Τί περιέχει η κλάση MonoBehaviour

Η κλάση **`MonoBehaviour`** είναι η θεμελιώδης βασική κλάση (base class) από την οποία κληρονομεί κάθε script που προσαρτάται ως component σε ένα GameObject στο Unity.

Χωρίς το `MonoBehaviour`, ένα script στη C# είναι απλώς μια απλή κλάση που δεν μπορεί να συνδεθεί με τον Inspector, δεν δέχεται drag-and-drop και δεν επικοινωνεί αυτόματα με τον κινητήρα του Unity.

1. Συναρτήσεις Κύκλου Ζωής (Event Messages / Lifecycle Callbacks)

Το Unity καλεί αυτόματα αυτές τις μεθόδους σε συγκεκριμένες χρονικές στιγμές της εκτέλεσης:

- **Αρχικοποίηση:**
  - `Awake()`: Μία φορά, μόλις δημιουργηθεί το component (ακόμα και αν είναι απενεργοποιημένο).
  - `OnEnable()`: Κάθε φορά που ενεργοποιείται το script ή το GameObject.
  - `Start()`: Μία φορά, στο πρώτο frame πριν από την πρώτη `Update()`.
- **Game Loop & Ανανέωση:**
  - `Update()`: Κάθε frame (μεταβλητός χρόνος, ιδανικό για inputs και γραφικά).
  - `FixedUpdate()`: Σε σταθερά χρονικά βήματα (fixed delta time, για υπολογισμούς φυσικής / Rigidbody).
  - `LateUpdate()`: Αμέσως μετά την `Update()` όλων των αντικειμένων (ιδανικό για κάμερα που ακολουθεί παίκτη).
- **Φυσική & Συγκρούσεις:**
  - `OnCollisionEnter / Stay / Exit`: Συγκρούσεις στερεών σωμάτων.
  - `OnTriggerEnter / Stay / Exit`: Διέλευση από trigger colliders (ζώνες ελέγχου).
- **Τερματισμός:**
  - `OnDisable()`: Όταν απενεργοποιείται το component.
  - `OnDestroy()`: Όταν διαγράφεται το GameObject ή αλλάζει η σκηνή.

2. Ενσωματωμένες Ιδιότητες (Built-in Properties)

Μέσα σε οποιοδήποτε `MonoBehaviour`, έχεις άμεση πρόσβαση σε βασικά στοιχεία του GameObject χωρίς να χρειάζεται να τα δηλώσεις:

- **`gameObject`**: Αναφορά στο ίδιο το αντικείμενο στο οποίο κάθεται το script.
- **`transform`**: Άμεση συντόμευση για το `GetComponent<Transform>()`.
- **`enabled`**: `true`/`false` για να ενεργοποιείς ή απενεργοποιείς το ίδιο το script.
- **`tag`** / **`name`**: Η ετικέτα και το όνομα του GameObject στο Hierarchy.

3. Μέθοδοι Διαχείρισης Components & Hierarchy

Παρέχει το API για εύρεση και σύνδεση άλλων components:

- `GetComponent<T>()`, `GetComponents<T>()`
- `GetComponentInChildren<T>()`, `GetComponentInParent<T>()`
- `TryGetComponent<T>(out T component)` (πιο ασφαλές και γρήγορο για αποφυγή null checks)

4. Μηχανισμός Coroutines

Το `MonoBehaviour` παρέχει τη δυνατότητα εκτέλεσης συναρτήσεων που διακόπτονται προσωρινά και συνεχίζουν σε επόμενα frames:

- `StartCoroutine(IEnumerator routine)`
- `StopCoroutine(...)` / `StopAllCoroutines()`

5. Σειριοποίηση & Σύνδεση με τον Inspector (Serialization)

Μέσω του `MonoBehaviour`, το Unity αναγνωρίζει τις μεταβλητές σου (`public` ή `[SerializeField] private`), τις εμφανίζει στο Inspector και αποθηκεύει τις τιμές τους απευθείας στο αρχείο της σκηνής ή του Prefab.

Πώς λειτουργεί ο μηχανισμός Serialization

1. **Την πρώτη φορά που βάζεις το script στον κύβο:**

   Το Unity διαβάζει τις προεπιλεγμένες τιμές που έγραψες στον κώδικα (π.χ. `upForce = 5f;`) και τις αποθηκεύει μέσα στο αρχείο της σκηνής (`.unity`) ως σειριοποιημένα δεδομένα (YAML data).

2. **Σε κάθε επόμενη αλλαγή στον κώδικα:**

   Αν ανοίξεις το script και αλλάξεις την τιμή σε `upForce = 1f;`, το Unity κάνει compile, αλλά **δεν πειράζει τις τιμές της σκηνής**. Όταν πατάς **Play**, το Unity αποσυμπιέζει (deserializes) τις αποθηκευμένες τιμές του Inspector και **αντικαθιστά (overwrites)** τις τιμές του C# script.

3. **Ο κανόνας προτεραιότητας:**

   Inspector (Αρχείο Σκηνής) > Αρχικές τιμές στον κώδικα C#

Η ιεραρχία κληρονομικότητας

Στην πραγματικότητα, το `MonoBehaviour` δεν τα περιέχει όλα μόνο του, αλλά τα κληρονομεί μέσα από μια βαθύτερη αλυσίδα κλάσεων:

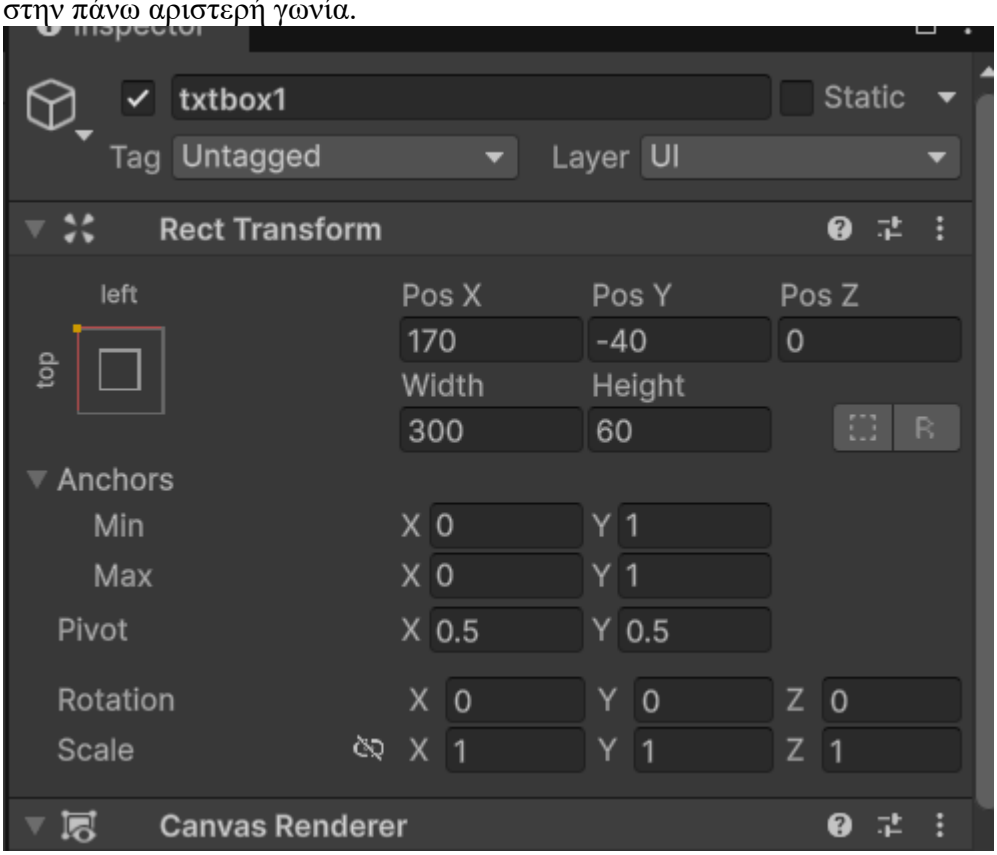
Plaintext

```
Object (UnityEngine.Object)
   └── Component
         └── Behaviour
               └── MonoBehaviour
                     └── ΤοΔικόΣουScript (CubeMover)
```

- **`Object`**: Επιτρέπει στο αντικείμενο να διαγράφεται με `Destroy()` και να αντιγράφεται με `Instantiate()`.
- **`Component`**: Του δίνει τη δυνατότητα να «κουμπώνει» πάνω σε GameObject (`GetComponent`).
- **`Behaviour`**: Προσθέτει το κουτάκι ενεργοποίησης/απενεργοποίησης (`enabled`).
- **`MonoBehaviour`**: Προσθέτει τα Lifecycle events (`Update`, `Start`, `Awake`), τις Coroutines και τη σειριοποίηση.

Παράδειγμα με Awake()
Να γίνει πρόγραμμα που να αρχικοποεί ένα text message με τη φράση initial message και κίτρινο χρώμα.

Βήματα στο περιβάλλον του Unity

1. **Δημιουργία του Text Box:**
   - Στο Hierarchy κάνε δεξί κλικ: **UI > Text - TextMeshPro**. Πάτησε **F2** πάνω στο νέο στοιχείο και μετονόμασέ το **ακριβώς** σε `txtbox1`. • Επίλεξε το **`txtbox1`** στο Hierarchy. Στο Inspector, κοίταξε το πρώτο component επάνω: **Rect Transform**. Κάνε κλικ στο τετραγωνάκι του **Anchor Presets** (το εικονίδιο με το σταυρόνημα πάνω αριστερά στο Rect Transform). Κράτα πατημένα τα πλήκτρα **`Shift + Alt`** (στο πληκτρολόγιο) και κάνε κλικ στο κουτάκι **πάνω αριστερά (top - left)**:Αυτό ορίζει τόσο το *Anchor* (σημείο πρόσδεσης) όσο και το *Pivot* στην πάνω αριστερή γωνία.
   - 

2. **Προσθήκη του Script:**
   - Δημιούργησε ένα νέο C# script με όνομα `TextInitializer.cs`.
   - Επικόλλησε τον παραπάνω κώδικα και αποθήκευσε (**Ctrl + S**).
   - Σύρε το script πάνω σε οποιοδήποτε ενεργό αντικείμενο της σκηνής (π.χ. στον Player ή σε ένα κενό GameObject / GameManager).
3. **Εκτέλεση:**

- Πάτησε **Play (▶)**: Αμέσως, πριν καν εμφανιστεί το πρώτο frame, η `Awake()` θα εντοπίσει το `txtbox1`, θα γράψει `Initial message` και θα αλλάξει τα γράμματα σε έντονο κίτρινο.

```
using UnityEngine;
using TMPro; // Απαραίτητο για το TextMeshPro

public class TextInitializer : MonoBehaviour
{
   private TextMeshProUGUI myTextBox;

   void Awake()
   {
      // 1. Αναζήτηση του GameObject στη σκηνή με βάση το ακριβές όνομα
      GameObject foundObject = GameObject.Find("txtbox1");

      // 2. Έλεγχος αν βρέθηκε το αντικείμενο
      if (foundObject != null)
      {
         // 3. Λήψη του TextMeshPro component από το αντικείμενο
//TextMeshProUGUI. Αυτό το component περιέχει τα πεδία text, color, fontSize κ.λπ.
         myTextBox = foundObject.GetComponent<TextMeshProUGUI>();

         // 4. Έλεγχος και εφαρμογή αρχικοποίησης (κείμενο & κίτρινο χρώμα)
         if (myTextBox != null)
         {
            myTextBox.text = "Initial message";
            myTextBox.color = Color.yellow;

            Debug.Log("Το txtbox1 αρχικοποιήθηκε επιτυχώς από την Awake!");
         }
         else
         {
            Debug.LogError("Βρέθηκε το αντικείμενο txtbox1, αλλά δεν έχει component TextMeshProUGUI!");
         }
      }
      else
      {
         Debug.LogError("Δεν βρέθηκε κανένα GameObject με όνομα txtbox1 στη σκηνή!");
      }
   }
}
```

Να γίνει πρόγραμμα που εμφανίζει έναν κύβο στο σημείο 10,30,0 με διαστάσεις 3,4,5 με χρώμα κόκκινο με χρήση της Awake() χωρίς να εισάγουμε εμείς κανένα αντικείμενο.

1. Στο παράθυρο **Project**, κάνε δεξί κλικ **Create > C# Script** και ονόμασέ το `CubeSpawner`.
2. Άνοιξε το αρχείο, επικόλλησε τον παραπάνω κώδικα και αποθήκευσε (**Ctrl + S**).
3. Στο **Hierarchy**, δημιούργησε ένα άδειο αντικείμενο: δεξί κλικ **Create Empty** (μετονόμασέ το π.χ. σε `SpawnerManager`).
4. Σύρε το script `CubeSpawner` πάνω στο `SpawnerManager`.
5. Πάτησε **Play (▶)**:
    - Πριν καν εμφανιστεί το πρώτο frame (`Start`), η `Awake()` θα γεννήσει τον κύβο στη θέση (10, 30, 0), με κλίμακα (3, 4, 5) και κόκκινο υλικό.

```
using UnityEngine;

public class CubeSpawner : MonoBehaviour
{
    void Awake()
    {
        // 1. Δημιουργία πρωταρχικού κύβου (Primitive Cube) στη σκηνή
        GameObject myCube = GameObject.CreatePrimitive(PrimitiveType.Cube);

        // Ονοματοδοσία του αντικειμένου στο Hierarchy
        myCube.name = "RedCustomCube";

        // 2. Τοποθέτηση στο σημείο (X: 0, Y: 1, Z: 30)
        myCube.transform.position = new Vector3(4f, 5f, 15f);

        // 3. Ορισμός διαστάσεων (Μήκος X: 3, Ύψος Y: 4, Βάθος Z: 5)
        myCube.transform.localScale = new Vector3(3f, 4f, 5f);

        // 4. Ανάκτηση του Renderer και εφαρμογή κόκκινου χρώματος. Ο renderer είναι που
βάζει το χρώμα (material)
        Renderer cubeRenderer = myCube.GetComponent<Renderer>();
        if (cubeRenderer != null)
        {
            cubeRenderer.material.color = Color.red;
        }

        Debug.Log("Ο κόκκινος κύβος δημιουργήθηκε επιτυχώς κατά την Awake!");
    }
}
```

- **Στην `Awake()`:** Δημιουργείς αντικείμενα (`Instantiate`, `CreatePrimitive`) και αρχικοποιείς τα εσωτερικά components του ίδιου του αντικειμένου (`GetComponent`).

- **Στην `Start()`:** Ψάχνεις αντικείμενα στη σκηνή (`GameObject.Find`, `FindFirstObjectByType`). Εκεί έχεις εγγύηση ότι όλοι έχουν περάσει από την `Awake()` τους και όλα τα απαραίτητα στοιχεία υπάρχουν ήδη στον κόσμο του παιχνιδιού.

Παράδειγμα με Start()

Έχουμε ένα αντικείμενο **`GameSettings`** (που ορίζει πόσες ζωές έχει ο παίκτης) και τον **`Player`** (που παίρνει τις ζωές του από το `GameSettings` μόλις ξεκινήσει το παιχνίδι). Με ποια σειρά πρέπει να γίνουν αυτά;

- Φτιάξτε ένα άδειο αντικείμενο στο Hierarchy (`Create Empty`) και ονόμασέ το **`SettingsManager`**. Ρίξε πάνω του το `GameSettings.cs`.

```
using UnityEngine;

public class GameSettings : MonoBehaviour
{
   public int startingLives;

   void Awake()
   {
      // Ετοιμάζει τις ρυθμίσεις πριν προλάβει να τις ζητήσει κανείς
      startingLives = 3;
      Debug.Log("GameSettings Awake: Οι ζωές ορίστηκαν σε 3.");
   }
}
```

- Φτιάξτε ένα δεύτερο άδειο αντικείμενο (ή στον κύβο σου) και ρίξε πάνω του το `PlayerLives.cs`.

```
using UnityEngine;

public class PlayerLives : MonoBehaviour
{
    private int currentLives;

    void Start()
    {
        GameSettings settings=null;
        // Εδώ είμαστε 100% σίγουροι ότι η Awake() του GameSettings έχει ήδη εκτελεστεί!

        ////Α τρόπος για να βρω το στοιχείο SettingsManager
        //βρες το πρώτο GameObject που έχει πάνω του το script GameSettings,
        // και επέστρεψέ μου το component του
        settings = FindFirstObjectByType<GameSettings>();

        //Β τρόπος για να βρω το στοιχείο SettingsManager από το παράθυρο Hierarchy
        /*GameObject settingsObj = GameObject.Find("SettingsManager");

        if (settingsObj != null)
        {
            // 2. Αντλεί το component GameSettings από μέσα του
            settings = settingsObj.GetComponent<GameSettings>();
        }*/


        if (settings != null)
        {
            currentLives = settings.startingLives;
            Debug.Log("PlayerLives Start: Ο παίκτης πήρε " + currentLives + " ζωές από το
GameSettings!");
        }
        else
        {
            Debug.LogError("Δεν βρέθηκε το GameSettings στη σκηνή!");
        }
    }
}
```

- Πάτησε **Play (▶)** και άνοιξε την καρτέλα **Console**:

Θα δεις πρώτα το μήνυμα της `Awake()` και έπειτα το μήνυμα της `Start()`, επιβεβαιώνοντας ότι η ανάγνωση έγινε με απόλυτη ασφάλεια.

Παράδειγμα με update

Κατασκευάζουμε έναν κόκκινο κύβο και δημιουργούμε ένα script c# και το ονομάζουμε CubeZMovement.cs. Δημιουργούμε έναν κύβο με το όνομα Player και προσθέτουμε τον ακόλουθο κώδικα. Φτιάχνουμε ένα empty GameObject και το ονομάζουμε GameManager και του ρίχνουμε πάνω τον κώδικα cs.

```
using UnityEngine;

public class CubeControllerManager : MonoBehaviour
{
    private Transform playerTransform;
  private int counter;

    [Header("Ρυθμίσεις Κίνησης")]
    public float speed = 2f; // 2 μέτρα ανά δευτερόλεπτο

    void Awake()
    {
    Debug.Log("Εκκίνηση της  Awake!");
           counter=0;
        // 1. Εντοπισμός του αντικειμένου Player στη σκηνή
        GameObject playerObj = GameObject.Find("Player");

        // 2. Αποθήκευση του Transform του
        if (playerObj != null)
        {
            playerTransform = playerObj.transform;
            Debug.Log("Ο κύβος εντοπίστηκε επιτυχώς από την Awake!");
        }
        else
        {
            Debug.LogError("Δεν βρέθηκε αντικείμενο με όνομα 'Player' στη σκηνή!");
        }
    }

    void Update()
    {
        counter++;
    Debug.Log("Εκκίνηση της  Update! "+counter+" η φορά");
        if (playerTransform != null)
        {
            // Υπολογισμός μετατόπισης για το τρέχον frame
            float distanceThisFrame = speed * Time.deltaTime;

            // Απευθείας πρόσθεση στη θέση του player κατά τον παγκόσμιο άξονα Z
            playerTransform.position += Vector3.forward * distanceThisFrame;
        }
    }
}
```

Εισαγωγή textbox για καταμέτρηση πτώσεων

Για να προσθέσεις ένα UI κείμενο (Text Box) που θα μετράει και θα εμφανίζει τις πτώσεις του παίκτη στην οθόνη, ακολουθείς μια καθαρή διαδικασία 4 βημάτων.

1.Δημιουργία UI Canvas & TextMeshPro στο Unity:

1. Στο παράθυρο **Hierarchy**, κάνε δεξί κλικ σε κενό χώρο και επίλεξε **UI > Text - TextMeshPro**.
2. Αν εμφανιστεί παράθυρο *TMP Importer*, πάτησε **Import TMP Essentials** και κλείσε το pop-up.
3. Το Unity θα δημιουργήσει αυτόματα ένα αντικείμενο **Canvas** και μέσα του ένα **Text (TMP)**, καθώς και ένα **EventSystem**.

2.Μορφοποίηση και Τοποθέτηση του Κειμένου:

1. Επίλεξε το αντικείμενο **Text (TMP)** στο Hierarchy (μπορείς να το μετονομάσεις σε `FailCounterText`).
2. Στο **Inspector** στα δεξιά:
   - Στο πεδίο **Text Input** γράψε ενδεικτικά: `Fails: 0`.
   - Ρύθμισε το **Font Size** (π.χ. σε `36` ή `40`).
   - Διάλεξε χρώμα (π.χ. λευκό ή κίτρινο) για να ξεχωρίζει από το φόντο.
3. Στο component **Rect Transform**:
   - Κάνε κλικ στο τετραγωνάκι του Anchor Presets (πάνω αριστερά στο Rect Transform).
   - Κράτα πατημένο το `Alt` (ή `Option`) και διάλεξε την **πάνω-αριστερή γωνία (top-left)**.
   - Δώσε **Pos X**: `160`, **Pos Y**: `-60` ώστε να απομακρυνθεί ελαφρώς από τα όρια της οθόνης.

3.Προσθήκη της λογικής του μετρητή στο Script:

Άνοιξε το `CubeMover.cs` και βεβαιώσου ότι περιλαμβάνει το `using TMPro;`, τη μεταβλητή για το κείμενο και τη συνάρτηση ενημέρωσης:

C#
```
using UnityEngine;
using UnityEngine.InputSystem;
using TMPro; // Απαραίτητο για το TextMeshPro

public class CubeMover : MonoBehaviour
{
    private Rigidbody rb;

    [Header("Δυνάμεις Άλματος")]
    public float upForce = 3.5f;
    public float forwardForce = 4.5f;
```

```
    [Header("UI Μετρητής")]
    public TextMeshProUGUI failText; // Αναφορά στο text box της οθόνης
    private int failCount = 0;       // Ο μετρητής πτώσεων

    private Vector3 startPosition;

    void Awake()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Start()
    {
        startPosition = transform.position;
        UpdateFailUI(); // Εμφάνιση αρχικού "Fails: 0"
    }

    void Update()
    {
        if (Keyboard.current != null &&
Keyboard.current.spaceKey.wasPressedThisFrame)
        {
            rb.linearVelocity = Vector3.zero;
            rb.AddForce(new Vector3(0, upForce, forwardForce),
ForceMode.Impulse);
        }

        // Αν πέσει κάτω από Y = -5
        if (transform.position.y < -5f)
        {
            Respawn();
        }
    }

    void Respawn()
    {
        // Αύξηση πτώσεων και ανανέωση οθόνης
        failCount++;
        UpdateFailUI();

        // Μηδενισμός ταχύτητας και επιστροφή στην αρχή
        rb.linearVelocity = Vector3.zero;
        rb.angularVelocity = Vector3.zero;
        transform.position = startPosition;
        transform.rotation = Quaternion.identity;
    }

    void UpdateFailUI()
    {
        if (failText != null)
        {
            failText.text = "Fails: " + failCount;
        }
    }
}
```

4.Σύνδεση του UI Text με τον Player:

1. Πάτησε **Ctrl + S** στον editor κώδικα και επίστρεψε στο Unity (περίμενε να ολοκληρωθεί το αυτόματο compile).
2. Επίλεξε τον **Player** (τον κύβο) στο Hierarchy.
3. Στο **Inspector**, στο component **Cube Mover (Script)**, εντόπισε το πεδίο **Fail Text** (θα γράφει *None (TextMeshPro - Text)*).
4. Σύρε με το ποντίκι το αντικείμενο **Text (TMP)** (ή `FailCounterText`) από το Hierarchy μέσα στο κουτάκι **Fail Text**.
5. Πάτησε **Play (▶)**: Κάθε φορά που ο κύβος πέφτει, επανέρχεται στην κορυφή και ο μετρητής αυξάνεται αυτόματα (`Fails: 1`, `Fails: 2`, κ.ο.κ.).